

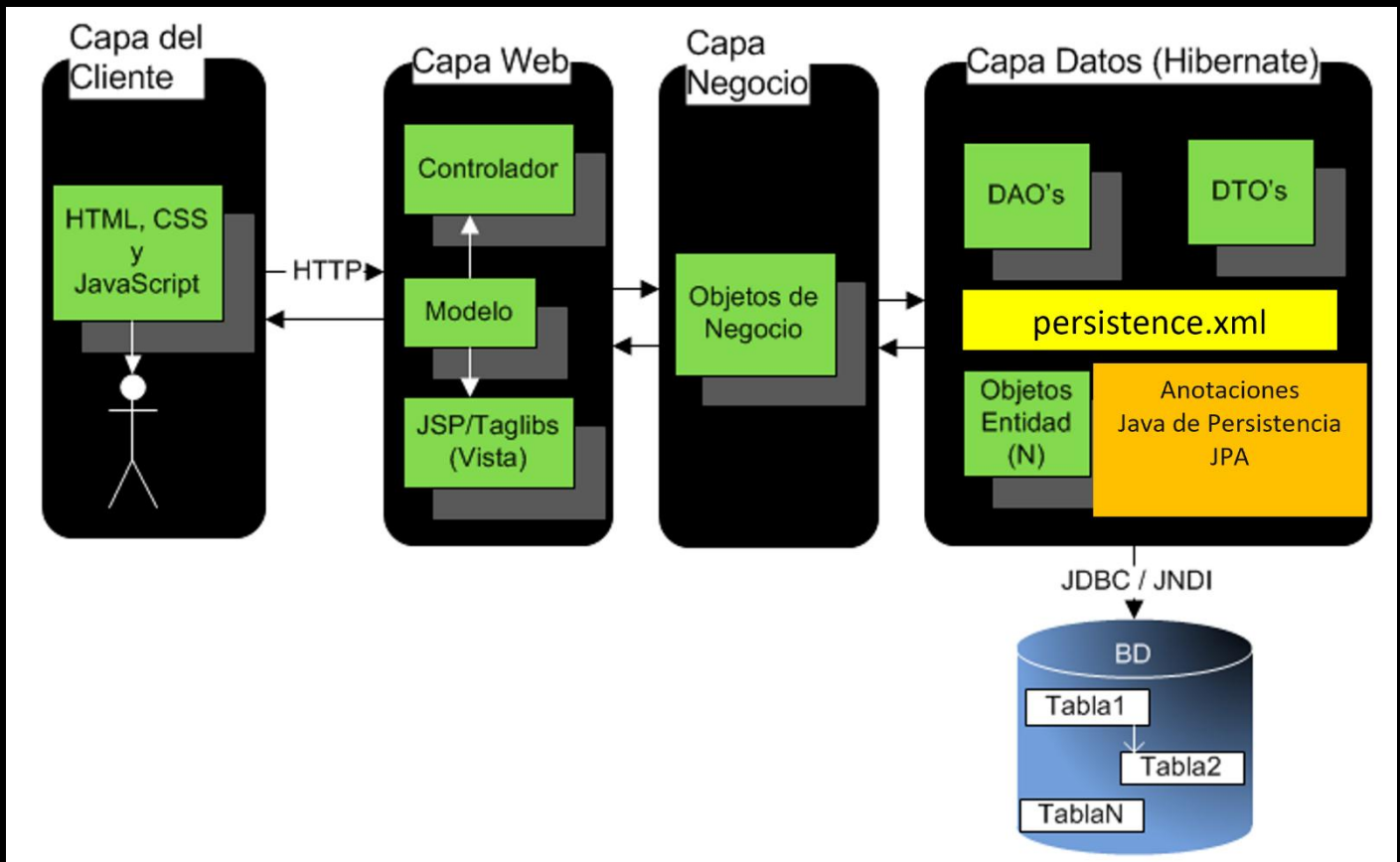


# APLICACIÓN WEB CON HIBERNATE

## Crear un Proyecto Web con Hibernate y JPA para Listar Personas

### Objetivo

Crear una aplicación web con arquitectura en capas usando Servlet, JSP, Hibernate y JPA, reutilizando el proyecto anterior para mostrar una lista de personas.



## ✨ Introducción

Vamos a construir una aplicación web basada en esta arquitectura:

```

Cliente (HTML, CSS, JS)
  ↓
Controlador (Servlet)
  ↓
Vista (JSP)
  ↓
Servicios (Clase PersonaService)
  ↓
DAO (Clase PersonaDAO)
  ↓
Entidad (Clase Persona)
  ↓
Base de Datos (Tabla persona en MySQL)
  
```

## 👣 Paso 1: Crear el nuevo proyecto web

1. Abre **Apache NetBeans**.
  2. Ve a **File > New Project**.
  3. Selecciona `Java with Maven > Web Application`.
  4. Haz clic en **Next**.
  5. Configura los campos:
    - o **Project Name:** `PersonasWebHibernateJPA`
    - o **Project Location:** ruta donde guardas tus cursos, por ejemplo:  
`C:\Cursos\Hibernate`
    - o **Group Id:** `gm`
    - o **Version:** `1.0`
    - o **Package:** déjalo vacío
  6. Haz clic en **Next**.
  7. Verifica que esté seleccionado:
    - o **Server:** `GlassFish` (ya configurado)
    - o **Java EE Version:** `Jakarta EE`
  8. Haz clic en **Finish**.
- 

## Paso 2: Agregar las dependencias del proyecto anterior

1. Cambiar el nombre del proyecto a `PersonasWebHibernateJPA`
    - a. Click derecho -> `Properties`
    - b. `Build` -> `Compile`
      - i. `Java Platform JDK 21`
      - ii. `Target Release: 21`
    - c. `IDE Settings`
      - i. Marcar la opción de `Compile on Save`
- 

## Paso 3: Agregar las dependencias del proyecto anterior

1. Abre el archivo `pom.xml` del proyecto anterior.
2. Copia el bloque completo de `<dependencies>...</dependencies>`.
3. Pega ese bloque dentro del nuevo `pom.xml` justo después de la dependencia de `jakarta.platform`.
4. Si tienes dependencias duplicadas (por ejemplo `<dependencies>` repetido), elimina la que sobre.
5. Guarda y haz clic derecho en el proyecto > **Build with Dependencies**.

 Código Completo `pom.xml`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```
<groupId>gm</groupId>
<artifactId>PersonasWebHibernateJPA</artifactId>
<version>1.0</version>
<packaging>war</packaging>
<name>PersonasWebHibernateJPA</name>

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <jakartaee>11.0.0-M1</jakartaee>
</properties>

<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>${jakartaee}</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.6.9.Final</version>
  </dependency>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.2.0</version>
  </dependency>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-api</artifactId>
    <version>2.24.3</version>
  </dependency>
  <dependency>
    <groupId>org.apache.logging.log4j</groupId>
    <artifactId>log4j-core</artifactId>
    <version>2.24.3</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
```

```

        <version>3.12.1</version>
        <configuration>
            <source>21</source>
            <target>21</target>
        </configuration>
    </plugin>
    <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.4.0</version>
    </plugin>
</plugins>
</build>
</project>

```

---

## Paso 4: Copiar las clases DAO y Entidad

1. Del proyecto anterior copia los paquetes:
  - o gm.dao
  - o gm.domain
2. Ve a Source Packages del nuevo proyecto.
3. Elimina los paquetes generados por defecto si no los necesitas.
4. Haz clic derecho > **Paste** y confirma.

 Archivo Persona.java:

```

package gm.domain;

import java.io.Serializable;
import jakarta.persistence.*;

@Entity
@Table(name = "persona")
public class Persona implements Serializable {

    private static final long serialVersionUID = 1L;

    @Column(name="id_persona")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    private Integer idPersona;

    private String nombre;

```

```
private String apellido;

private String email;

private String telefono;

public Persona(){

}

public Integer getIdPersona() {
    return idPersona;
}

public void setIdPersona(Integer idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
```

```
        this.telefono = telefono;
    }

    @Override
    public String toString() {
        return "Persona{" + "idPersona=" + idPersona + ", nombre=" + nombre + ", apellido=" + apellido +
            ", email=" + email + ", telefono=" + telefono + '}';
    }
}
```

#### Archivo PersonaDAO.java:

```
package gm.dao;

import gm.domain.Persona;
import jakarta.persistence.*;

import java.util.List;

public class PersonaDAO implements AutoCloseable {

    private final EntityManagerFactory emf;
    private final EntityManager em;

    public PersonaDAO() {
        this.emf = Persistence.createEntityManagerFactory("HibernatePU");
        this.em = emf.createEntityManager();
    }

    public List<Persona> listar() {
        String hql = "SELECT p FROM Persona p";
        return em.createQuery(hql, Persona.class).getResultList();
    }

    public void imprimirPersonas() {
        listar().forEach(p -> System.out.println(" " + p));
    }

    public void insertar(Persona persona) {
        var tx = em.getTransaction();
        try {
            tx.begin();
            em.persist(persona);
            tx.commit();
        }
    }
}
```

```
    } catch (Exception e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw e; // se relanza la excepción para que la maneje el llamador si quiere
    }
}

public void modificar(Persona persona) {
    var tx = em.getTransaction();
    try {
        tx.begin();
        em.merge(persona);
        tx.commit();
    } catch (Exception e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw e;
    }
}

public Persona buscarPersonaPorId(Persona persona) {
    try {
        return em.find(Persona.class, persona.getIdPersona());
    } catch (Exception e) {
        throw new PersistenceException("Error al buscar persona por ID: " + persona.getIdPersona(),
e);
    }
}

public void eliminar(Persona persona) {
    var tx = em.getTransaction();
    try {
        tx.begin();
        em.remove(em.merge(persona));
        tx.commit();
    } catch (Exception e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw new PersistenceException("Error al eliminar persona con ID: " + persona.getIdPersona(),
e);
    }
}
```



```

@Override
public void close() {
    if (em.isOpen()) {
        em.close();
    }
    if (emf.isOpen()) {
        emf.close();
    }
}
}
}

```

---

## Paso 5: Agregar archivo `log4j2.properties`

1. Copia el archivo `log4j2.properties` desde el proyecto anterior.
2. Pégallo en esta ruta:

```
src/main/resources/log4j2.properties
```

3. El contenido del archivo debe ser:

```

# SQL ejecucion
logger.hibernate.name = org.hibernate.SQL
logger.hibernate.level = debug

# JDBC pamaros binding
logger.jdbc-bind.name=org.hibernate.orm.jdbc.bind
logger.jdbc-bind.level=trace

```

Este archivo es importante para ver las sentencias SQL que se ejecutan, incluyendo los parámetros utilizados por Hibernate. 🔍

---

## Paso 6: Agregar archivo `persistence.xml`

1. Copia el archivo `persistence.xml` desde:

```
src/main/resources/META-INF/persistence.xml
```

2. Pega el archivo en la misma ruta del nuevo proyecto.
3. Si ya existe uno vacío, elimínalo y reemplázalo por el que ya tiene la configuración.

### Código completo archivo persistence.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
        https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

    <persistence-unit name="HibernatePU" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <class>gm.domain.Persona</class>
        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=true&useTimezone=true&serverTimezone=UTC"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password" value="admin"/>
            <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
        </properties>
    </persistence-unit>
</persistence>
```

---

## Verificar estructura final

Al finalizar, tu proyecto debe tener esta estructura:

```
personas-web-hibernate-jpa
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── gm
│   │   │   │   ├── dao
│   │   │   │   └── domain
│   │   └── resources
│   │       ├── META-INF
│   │       │   └── persistence.xml
│   │       └── log4j2.properties
└── pom.xml
```

---

## Conclusión

¡Excelente trabajo! Ya tienes listo tu proyecto web base para mostrar el listado de personas. A partir de este punto, vas a construir las capas de servicios y la capa web con JSPs y Servlets.

Aún no debes ejecutar el proyecto, ya que vamos a crear la capa de servicio y presentación (web) en la siguiente lección. ¡Continuemos!

---

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

**Ing. Marcela Gamiño e Ing. Ubaldo Acosta**

**Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)**